

Concurrency Control: 2PL Variants

01 QUICK REVISION OF BASIC 2PL

1. Growing Phase

 Lock

- Acquire locks as required ✓
- **Cannot release** any locks ✗
- Lock point: Moment all locks are obtained

2. Shrinking Phase

 Unlock

- Release locks as needed ✓
- **Cannot acquire** new locks ✗
- Once first lock is released, phase starts

Guaranteed Property:

✓ **Conflict Serializability** is guaranteed by Basic 2PL.

Unsolved Vulnerabilities:

✗ **Irrecoverable schedules** | ✗ **Cascading rollback**
✗ **Deadlocks possible** | ✗ **Starvation possible**

02 STRICT 2PL (STRICT TWO-PHASE LOCKING)

Definition ★: Strict 2PL = Basic 2PL + All **Exclusive (X)** locks are held until **COMMIT or ABORT**. Shared (S) locks may be released during the normal shrinking phase.

Execution Flow & Mechanics

Suppose transaction T1 writes data item A, and T2 wishes to read item A:

T1: X(A) → Read(A) → Write(A) → COMMIT → Unlock(A)

Lock)

↓ (T1 Releases

T2: [WAIT] → S(A) → Read(A)

Result: T2 is forced to read the final committed database state, preventing dirty reads.

03 PROBLEMS REMOVED BY STRICT 2PL

1. Cascading Rollback Removed ✓

In basic 2PL, if T1 fails, dependent transactions (T2, T3) reading T1's uncommitted writes must also roll back in a cascade.

Strict 2PL prevents T2/T3 from reading item A until T1 commits/aborts, ensuring **Cascadeless Schedules**.

2. Irrecoverability Removed ✓

In basic 2PL, T2 might read T1's uncommitted write and commit. If T1 then aborts, T2 cannot roll back (Irrecoverable).

Strict 2PL blocks T2 until T1 commits, guaranteeing **Strict Recoverability**.

 **MEMORY TRICK: STRICT = X-locks stay STRICTly held until COMMIT / ABORT.**

04 RIGOROUS 2PL (RIGOROUS TWO-PHASE LOCKING)

Definition ★: Rigorous 2PL holds **BOTH Shared (S) and Exclusive (X) locks** until the transaction **COMMITTS or ABORTS**.

Strict 2PL Rule

- **X-Locks:** Held until COMMIT/ABORT
- **S-Locks:** May be released before commit
- *Less restrictive concurrency mode*

Rigorous 2PL Rule

- **X-Locks:** Held until COMMIT/ABORT
- **S-Locks:** Held until COMMIT/ABORT
- *More restrictive concurrency mode*

🧠 **MEMORY TRICK:** Strict → Holds X-Locks only | Rigorous → Holds X + S Locks

05 UNRESOLVED ISSUES IN STRICT & RIGOROUS 2PL

Deadlock is NOT Removed ✗

Neither Strict nor Rigorous 2PL prevents circular wait conditions.

T1 holds X(A), requests X(B) → [WAIT]

T2 holds X(B), requests X(A) → [WAIT]

Result: Circular Wait (Deadlock)

Starvation is NOT Removed ✗

A transaction may wait indefinitely if other transactions repeatedly obtain compatible locks on the required data item first.

Strict and Rigorous 2PL do not enforce queue fairness on lock requests.

06 CONSERVATIVE 2PL (STATIC 2PL)

Definition ★: Conservative 2PL requires a transaction to **acquire ALL locks it will ever need before execution begins**.

Operational Protocol & Deadlock Prevention

```
Request ALL locks (A, B, C) together → [Available?]
├→ YES: Acquire ALL locks → Execute → Release Locks
└→ NO: Acquire NONE → Wait without holding partial locks
```

Why is Conservative 2PL Deadlock-Free?

Deadlocks require a hold-and-wait condition. Since Conservative 2PL forces a transaction to acquire all locks upfront or hold none at all, circular wait is mathematically impossible.

Practical Limitation: Requires prior knowledge of all data items a transaction will access. In dynamic databases, future access patterns are often unpredictable, making Conservative 2PL impractical for general use.

07 COMPREHENSIVE COMPARISON MATRIX

Property / Feature	Basic 2PL	Strict 2PL	Rigorous 2PL	Conservative 2PL
Conflict Serializability	✓ Yes	✓ Yes	✓ Yes	✓ Yes
X-Lock Held Until Commit	✗ No	✓ Yes	✓ Yes	Depends
S-Lock Held Until Commit	✗ No	✗ No	✓ Yes	Depends
Guarantees Recoverability	✗ No	✓ Yes	✓ Yes	Depends
Guarantees Cascadelessness	✗ No	✓ Yes	✓ Yes	Depends
Deadlock-Free	✗ No	✗ No	✗ No	✓ Yes
Starvation-Free	✗ No	✗ No	✗ No	✗ No
Main Limitation	Cascading Aborts	Lower Concurrency	Lowest Concurrency	Requires Prior Knowledge

08 COMPETITIVE EXAM TRAPS & MULTIPLE-CHOICE Q&A

Q1: Does Basic 2PL guarantee serializability?

YES ✓ (Guarantees conflict serializability).

Q2: Does Basic 2PL guarantee recoverability?

NO ✗ (Can produce dirty reads and irrecoverable schedules).

Q3: Does Strict 2PL prevent cascading rollbacks?

YES ✓ (Forces reading only committed data).

Q4: Does Strict 2PL eliminate deadlocks?

NO ✗ (Deadlocks can still occur).

Q5: Which locks are retained until commit in Rigorous 2PL?

Both Shared (S) and Exclusive (X) locks ✓.

Q6: Which 2PL variation guarantees deadlock freedom?

Conservative (Static) 2PL ✓.

Q7: Which protocol is more restrictive: Strict or Rigorous?

Rigorous 2PL ✓.

Q8: Does Conservative 2PL prevent starvation?

NO ✗ (Transactions may wait indefinitely if resources are constantly busy).

09 GOLDEN REVISION SUMMARY

Basic 2PL Serializability ✓ Deadlock / Cascade ✗	Strict 2PL X-Locks till Commit Cascadeless ✓	Rigorous 2PL S + X till Commit Strict & Cascadeless ✓	Conservative 2PL ALL locks before start Deadlock-Free ✓
---	---	--	--